

¿Qué es la IA generativa?

2

¿Qué diferencia la IA Generativa de otros tipos de IA?

- **Modelos Generativos:** Crean contenido nuevo basándose en patrones aprendidos (texto, imágenes, música, código).
- **No Generativos:** Se limitan a clasificar, analizar o responder basándose en datos existentes, sin producir contenido original. Dan como respuesta un valor numérico o una de las opciones predeterminadas.

Ejemplo:

Poner una nota a un ensayo no es necesariamente IA Generativa.

Argumentar la evaluación sí es IA Generativa.

2. ¿Qué es la IA Generativa?

¿Qué tareas puede realizar IA Generativa?

T2T

¿Qué imagen podría representar este objetivo?
Entender las capacidades de la IA Generativa para poder proponer nuevas soluciones innovadoras.
La descripción debe ser breve y creativa.

Una lámpara encendida flotando sobre una red de circuitos brillantes, de la cual emergen pequeñas figuras y símbolos que representan ideas y creatividad en forma de hologramas.

Otros casos:

- Generación de audio
- Generación de video
- Generación de código

T2I

Por favor, crea la imagen que has descrito.

Imagen creada



Imagen creada por ChatGPT

3

¿Qué técnicas de análisis de texto se usaban antes de la IA generativa y por qué tenemos que conocerlas?

¿Por qué siguen siendo clave los métodos clásicos en la era de la IA Generativa?

Piedras angulares del conocimiento estructurado

Técnicas como NER (reconocimiento de entidades) permiten identificar nombres, fechas o lugares. Son la base para organizar el texto antes de generar contenido con IA.

IA Generativa ≠ Magia: necesita buenos datos

Los modelos generativos aprenden mejor cuando los alimentamos con información limpia y bien estructurada. Aquí brillan los métodos clásicos de NLP.

RAG: lo mejor de dos mundos

La generación aumentada con recuperación (RAG) combina IA generativa con técnicas tradicionales como TF-IDF o embeddings para encontrar y usar la información más relevante.

Más barato, más simple, igual de útil (a veces)

No todo necesita IA generativa. Técnicas tradicionales como la extracción de palabras clave o el análisis semántico resuelven muchos problemas con menos recursos.

Ejemplo de NER

Persona	Localización	Organización	Tiempo
María	trabaja	en	INECO
en	Valencia	desde	2015
INECO	y	Ministerio de Transporte	
firmaron	el	contrato	en
Madrid	el	25 de mayo de 2003	

Ejercicio

Por favor, escribid un texto breve (3-5 frases) vuestra opinión sobre los aspectos negativos y positivos de la IA Generativa.

- **¿Qué es un corpus?**

Un corpus es un conjunto de textos que usamos para analizar el lenguaje.

- **Limpieza**

Quitar tildes y caracteres especiales

Quitar signos de puntuación

Pasar a minúscula

- **Tokenización**

Crear lista de palabras

- **Bag of words**

Contar la ocurrencia de cada palabra en la lista

"Me encanta usar IA, la IA es increíble",
"IA siempre da respuestas brillantes, es perfecta",

"me encanta usar ia la ia es increíble",
"ia siempre da respuestas brillantes es perfecta",

[me, encanta, usar, ia, la, ia, es, increíble, ia,
siempre, da, respuestas, brillantes, es,
perfecta]

[me:1, encanta:1, usar:1, ia:3, la:1, es:2, increíble:1,
siempre:1, da:1, respuestas:1, brillantes:1,
perfecta:1]



- ¿Qué son las stop words?

Son palabras muy frecuentes que, por sí solas, aportan poco significado.

Ejemplos comunes: "el", "la", "de", "que", "y", "pero", "no", "muy", "con"...

Eliminarlas ayuda a centrarse en palabras importantes.

- ¿Siempre conviene la eliminación?

En algunos casos, eliminar estas palabras puede distorsionar el significado del texto:

"fiable" → puede indicar algo positivo

"no fiable" o "poco fiable" → claramente negativo

- Depende del enfoque

En modelos básicos, como el Bag of Words, quitar stop words suele ser útil y efectivo.

En enfoques más avanzados como embeddings o modelos de lenguaje, es mejor conservarlas.

En modelos intermedios (como TF-IDF) conviene decidir según el objetivo.



[me, encanta, usar,
ia, la, ia, es, increíble,
ia, siempre, da,
respuestas,
brillantes, es,
perfecta]

[encanta, usar,
ia,ia, increíble, ia,
siempre, da,
respuestas,
brillantes, perfecta]



Lematización

- Devuelve la **forma básica del diccionario (el lema)**
- Tiene en cuenta el **significado y la gramática**
- Es **más precisa**, pero también **más lenta**

Ejemplos:

- “hablando” → **hablar**
- “ingenieros” → **ingeniero**

Stemming

- Recorta las palabras a su **raíz aproximada**, sin entender el contexto
- Funciona con reglas mecánicas
- Es **más rápido**, pero puede dar resultados incorrectos o artificiales

Ejemplos:

- “hablando” → **habl**
- “ingenieros” → **ingenier**



[me, encanta, usar, ia,
la, ia, es, increíble, ia,
siempre, da,
respuestas, brillantes,
es, perfecta]

[yo, encantar, usar,
ia, ia, increíble, ia,
siempre, dar,
respuesta, brillante,
perfecta]

[illegible]

¿Qué es TF-IDF?

TF-IDF es una técnica que mide **cuán importante es una palabra en un texto, comparándola con el resto del corpus**.

TF (Term Frequency) cuenta cuántas veces aparece una palabra en un documento.

IDF (Inverse Document Frequency) reduce la importancia de las palabras que aparecen en muchos documentos.

El resultado **destaca palabras frecuentes en un texto, pero poco comunes en el resto**, lo que ayuda a identificar términos relevantes.

$$TF(t, d) = \frac{\text{número de veces que aparece } t \text{ en } d}{\text{número total de palabras en } d}$$

Donde:

- t es el término (la palabra)
- d es el documento

$$IDF(t) = \log \left(\frac{1 + N}{1 + df(t)} \right) + 1$$

donde:

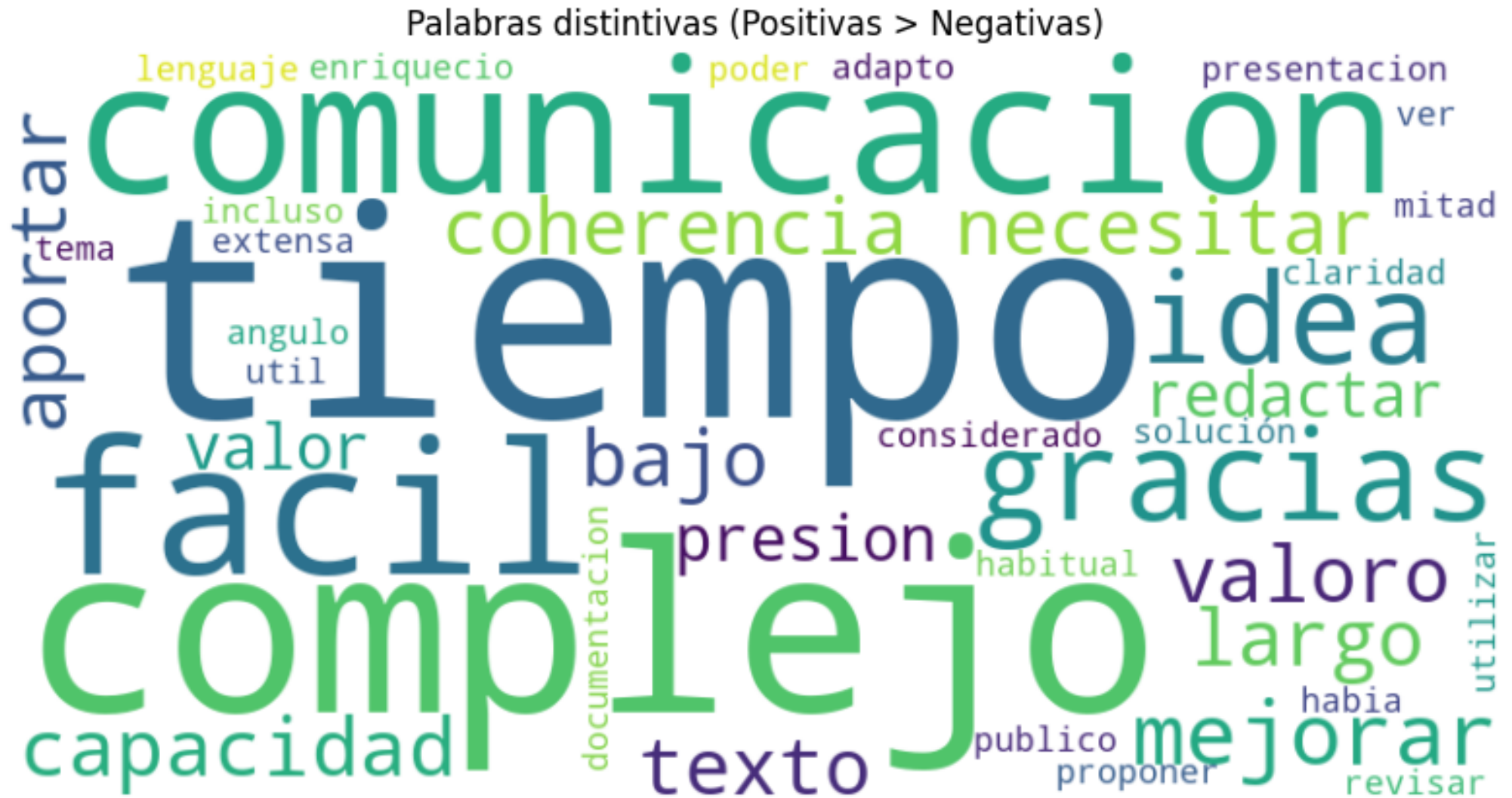
- N = número total de documentos
- $df(t)$ = número de documentos donde aparece la palabra t

Corpus:

"ChatGPT es increíble, ChatGPT es genial, lo uso todo el rato",
"Me encanta usar ChatGPT, la IA es increíble",
"ChatGPT siempre da respuestas brillantes, es perfecto",
"ChatGPT me encanta, es extraordinario trabajar con IA",
"ChatGPT es un desastre, solo da errores, la IA no nos va a sustituir",
"La IA no sirve, ChatGPT es completamente inútil",
"ChatGPT responde con vaguedades superficiales",
"ChatGPT solo da respuestas con errores, es inútil",
"ChatGPT es un modelo de lenguaje desarrollado por OpenAI"

Cálculo de TF-IDF para opinión 1 (algunas palabras):

brillantes	→ TF = 0.000 IDF = 2.609 TF-IDF = 0.000
chatgpt	→ TF = 0.333 IDF = 1.000 TF-IDF = 0.333
desarrollado	→ TF = 0.000 IDF = 2.609 TF-IDF = 0.000
desastre	→ TF = 0.000 IDF = 2.609 TF-IDF = 0.000
errores	→ TF = 0.000 IDF = 2.204 TF-IDF = 0.000
genial	→ TF = 0.167 IDF = 2.609 TF-IDF = 0.435
ia	→ TF = 0.000 IDF = 1.693 TF-IDF = 0.000
increíble	→ TF = 0.167 IDF = 2.204 TF-IDF = 0.367
inútil	→ TF = 0.000 IDF = 2.204 TF-IDF = 0.000
rato	→ TF = 0.167 IDF = 2.609 TF-IDF = 0.435
responde	→ TF = 0.000 IDF = 2.609 TF-IDF = 0.000
usar	→ TF = 0.000 IDF = 2.609 TF-IDF = 0.000
uso	→ TF = 0.167 IDF = 2.609 TF-IDF = 0.435



Palabras distintivas (Negativas > Positivas)



¿Qué es un *embedding*?

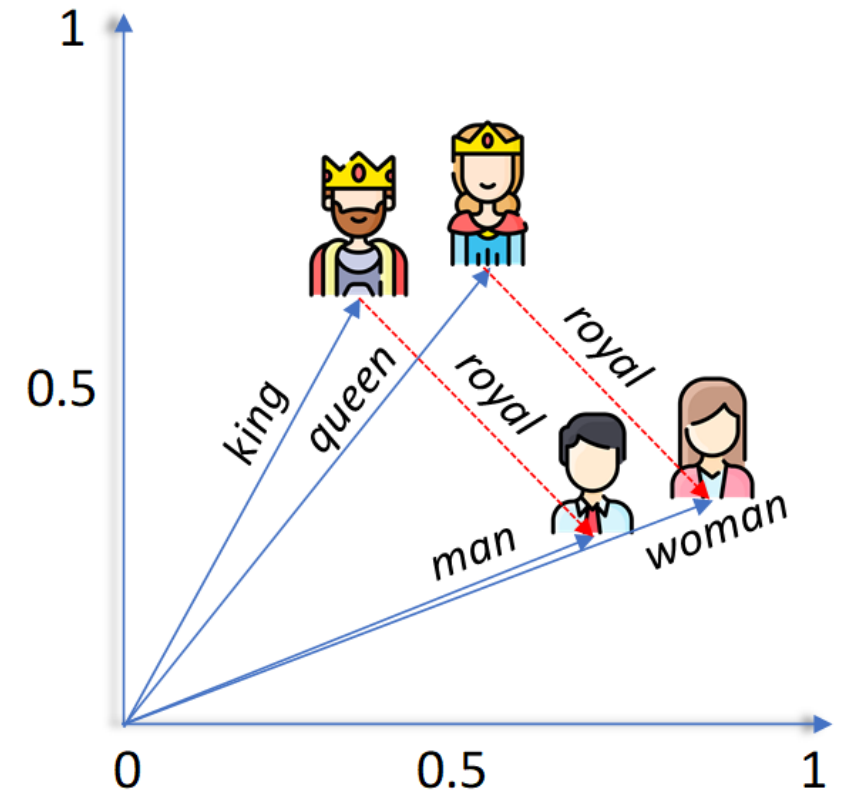
Un **embedding** es una forma de representar texto con números que capturan su contenido y significado.

Tipos de representaciones:

- **Por palabra:** Cada palabra (o *token*) se transforma en un vector (una lista de números).
Ejemplo: "bueno" → [0.2, 0.8, -0.1, ...]
- **Por texto completo:** Se crea un único vector para toda una frase o párrafo.
Ejemplo: "Me gusta usar ChatGPT en el trabajo." → [0.1, 0.6, -0.3, ...]
- **Por frecuencia:** En métodos como TF-IDF, el texto se representa con tantos números como palabras del vocabulario.
Ejemplo: [0, 0.4, 0.2, 0, 0.7, ...]

¿Para qué sirve?

- Los embeddings de texto basados en frecuencia permiten a las máquinas **comparar textos**.
- Los embeddings de palabra permiten a la máquina "entender" el texto y los embeddings avanzados de texto completo permiten entender el contexto.



Fuente: towardsdatascience.com

¿Cómo medir si dos textos se parecen?

1. Similitud del coseno

Compara la orientación entre dos vectores, no sus longitudes.

2. ¿Y si el texto no es un solo vector?

Si cada palabra del texto se convierte en un **vector** → el texto completo es una **matriz**.

Para comparar textos, hay que reducir cada matriz a un solo vector.

3. ¿Cómo se reduce?

Lo más común: hacer la media de todos los vectores de las palabras.

4. ¿Y si los textos tienen distinta longitud?

No se pueden comparar matrices de tamaños distintos → se usa:

- ✂ Truncate: cortar textos largos.
- ⊕ Padding: rellenar textos cortos con tokens vacíos.

Aunque al final hagamos una media, la longitud sí importa:

👉 Textos largos pueden tener más “palabras de adorno”, saludos, rodeos...

👉 Eso puede diluir el vector final y falsear la similitud.

```
■ Texto: Hola, ¿cómo estás? Me gustaría hablar sobre inteligencia artificial, me interesa mucho.
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'Hola' → Vector: [-0.09495 -1.436 -2.165 1.0912 -1.6133]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'estás' → Vector: [-3.1265 -2.4515 0.92381 0.17106 -0.43178]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'gustaría' → Vector: [-1.8672 -0.92822 -0.35389 0.011666 -0.84668]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'hablar' → Vector: [ 0.077984 2.6901 -2.3481 -1.183 -1.4443]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'inteligencia' → Vector: [ 0.97444 1.2938 -0.68326 -1.5929 0.14744]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'artificial' → Vector: [ 0.078362 0.55421 0.27935 -1.1564 1.6534]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'interesa' → Vector: [-1.2588 -0.75155 0.25812 -0.23713 0.56607]...

■ Matriz de vectores (shape: 7x300)
✂Truncate: Cortando a 6 vectores

■ Vector final (media): [-0.65964395 -0.04626827 -0.724515 -0.4430623 -0.4225367]...

■ Texto: Inteligencia artificial es un tema fascinante que cambia el mundo, leo mucho sobre ello.
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'Inteligencia' → Vector: [-0.73282 1.7708 0.8346 -1.1534 -0.29512]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'artificial' → Vector: [ 0.078362 0.55421 0.27935 -1.1564 1.6534]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'tema' → Vector: [-1.637 -2.2364 -2.6325 -3.9785 2.3823]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'fascinante' → Vector: [-0.79128 -0.15934 -2.1299 1.8077 0.31674]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'cambia' → Vector: [-2.2854 0.59628 -0.10756 0.36064 -0.057237]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'mundo' → Vector: [ 0.57741 -1.5903 -4.1834 0.20722 3.3015]...
Se muestran los primeros 5 componentes de los 300.
■ Palabra: 'leo' → Vector: [ 0.37802 -5.8303 -0.81568 -2.6644 -1.4825]...

■ Matriz de vectores (shape: 7x300)
✂Truncate: Cortando a 6 vectores

■ Vector final (media): [-0.7984546 -0.1774583 -1.323235 -0.6521232 1.2169304]...

▲ Similitud del coseno entre textos: 0.490
```

Detalle de cálculo de similitud de coseno

Vectores:

Supongamos que tenemos:

$$A = [1, 2] \quad \text{y} \quad B = [2, 3]$$

Paso 1: Producto punto $A \cdot B$

$$1 \cdot 2 + 2 \cdot 3 = 2 + 6 = 8$$

Paso 2: Norma (longitud) de cada vector

$$\|A\| = \sqrt{1^2 + 2^2} = \sqrt{1 + 4} = \sqrt{5} \approx 2.236$$

$$\|B\| = \sqrt{2^2 + 3^2} = \sqrt{4 + 9} = \sqrt{13} \approx 3.606$$

Paso 3: Similitud del coseno

$$\cos(\theta) = \frac{8}{2.236 \cdot 3.606} \approx \frac{8}{8.062} \approx 0.993$$

Resultado:

Similitud del coseno ≈ 0.993

Esto indica que **los vectores son muy similares** en dirección, casi paralelos.

$$\text{Similitud del coseno} = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Fundamentos Técnicos de los Modelos Generativos de Texto

4

¿Qué es un LLM?

LLM significa **Large Language Model** (Modelo de Lenguaje Grande).

- Es un programa entrenado con muchísimo texto para poder **predecir la siguiente palabra** de una frase.
- Por eso puede **responder preguntas, escribir textos, resumir, traducir...**

¿Predecir la siguiente palabra? ¿No será predecir un texto?

En realidad, un LLM solo sabe generar una palabra a la vez (o más exactamente, un "token", que puede ser una sílaba o fragmento de palabra). Pero lo hace tan rápido y con tanta precisión, que parece que genera frases enteras de golpe.

¿Qué quiere decir GPT de Chat GPT?

- “**Generative**”: genera texto, no se limita a clasificar o responder sí/no.
- “**Pre-trained**”: fue entrenado con grandes cantidades de texto antes de poder usarse.
- “**Transformer**”: es la arquitectura (tipo de red neuronal) que usa, inventada por Google en 2017.

¿Y Claude, Deepseek o Gemini también son GPT?

Sí en lo técnico: todos son modelos generativos, preentrenados y usan transformers.

No en el nombre: GPT es una familia concreta de modelos creada por OpenAI. Es una marca comercial.

¿Transformers?

- Es la **arquitectura** o el “esqueleto” que usan los LLMs para funcionar.
- Fue inventado por Google en 2017 y cambió completamente el campo de la IA.
- Su punto fuerte es algo llamado “**atención**”: permite que el modelo se **fije en las palabras importantes** de un texto, aunque estén lejos unas de otras.
- Es muy eficiente para aprender el **significado de frases largas** o textos complejos.

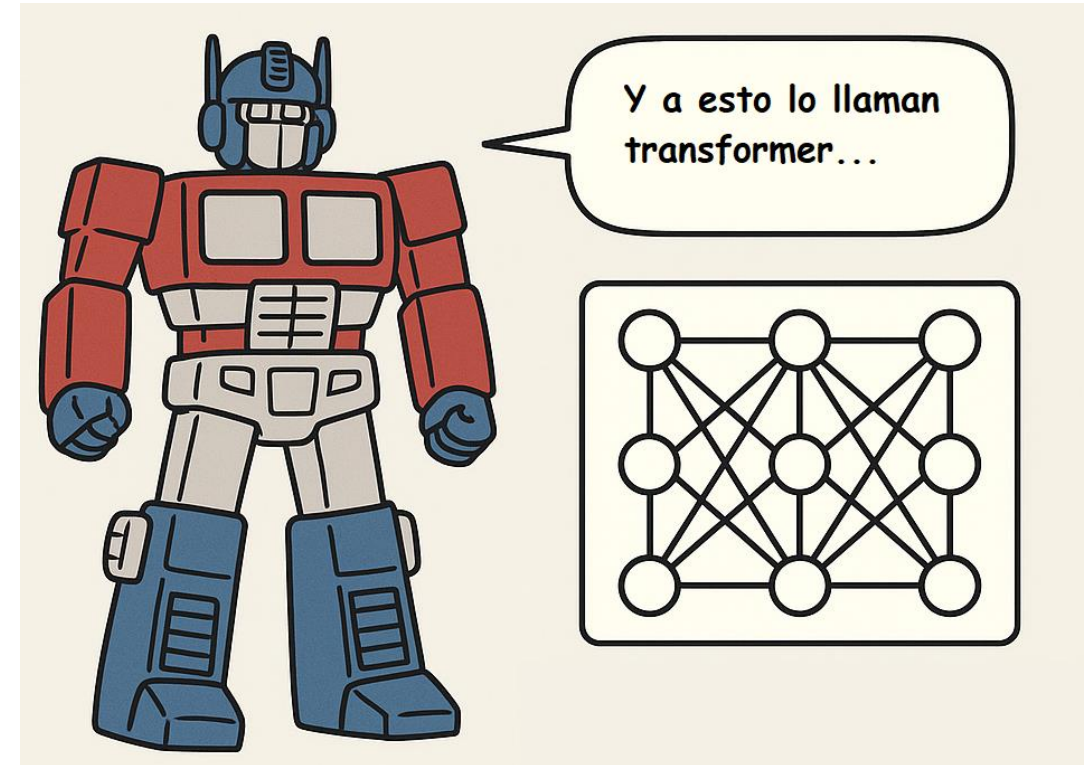


Imagen creada por ChatGPT

Conceptos clave para entender los LLM



Sets de Datos: Los modelos se entrenan con datos textuales como preguntas-respuestas, lógica o patrones en corpus grandes y diversos.



Ventana de Contexto: Representa la cantidad de información previa que el modelo puede considerar en su razonamiento. Cuanto mayor sea la ventana, más contexto puede procesar.



Tokens:

- *Tokens de entrada:* Fragmentos del texto inicial que el modelo procesa.
- *Tokens de salida:* Fragmentos generados por el modelo, que se construyen uno a uno.



Temperatura: Controla la creatividad de las respuestas; baja temperatura produce resultados más precisos y alta temperatura genera respuestas más variadas.

**De Modelo a Producto: ¿cómo crear una
solución basada en IA Generativa?**

¿En qué se diferencia un modelo de IA Generativa de una aplicación con IA Generativa?

El modelo es solo **el motor**.

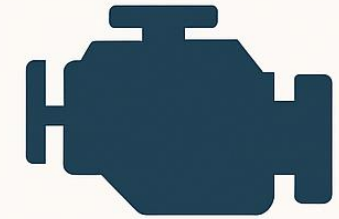
No se puede lanzar el modelo como si fuera una aplicación. Un modelo no es un producto. Es **una pieza**. Y, además, **una que no arranca sola**.

¿Qué es realmente un modelo?

Es el **motor inteligente**: puede razonar, generar, predecir...

Existen motores “comerciales” (via APIs como **Bedrock de AWS**) y modelos **open source** que pueden descargarse en el equipo con herramientas como **Ollama, Hugging Face, LangChain**.

Difieren en potencia, coste de consumo, aplicabilidad. Pero en todos los casos, **no basta con tener el motor**.



MODELO



APLICACIÓN

Imagen creada por ChatGPT

¿Qué elementos tiene la aplicación aparte de modelo?

Elementos parecidos a MLOps tradicional:

Infraestructura	Se necesita una base técnica para alojar modelos, APIs, datos y mantenerlos operativos.
Escalabilidad	Ambos requieren capacidad de servir a múltiples usuarios o procesos al mismo tiempo.
Orquestación	Es necesario definir flujos : cuándo se activa el modelo, con qué datos, y qué hacer con la respuesta.
Supervisión y control	Ambos tipos de sistemas deben garantizar seguridad, calidad, trazabilidad y cumplimiento de normas.
Gestión de datos relacionales	Tanto en ML tradicional como en GenAI se utilizan bases de datos clásicas para usuarios, logs, registros, etc.
Versionado (modelos, datos)	Se versionan modelos, datos y procesos para poder reproducir y auditar resultados.

¿Qué elementos tiene la aplicación aparte de modelo?

Elementos diferentes a MLOps tradicional:

Entrenamiento del modelo	En GenAI se parte de modelos ya entrenados (como GPT, LLaMA...). No entrenas desde cero: haces fine-tuning , RAG o prompting .
Input/Output	El input no es un vector o número, sino texto abierto , preguntas, instrucciones. El output también es texto libre .
Evaluación del modelo	No hay métricas claras como accuracy o F1. Hay que evaluar coherencia , utilidad , tono , etc.
Interfaz de usuario	La interacción suele ser conversacional (chat, voz, texto libre) y la interfaz es más importante.
Gestión de datos vectoriales	En GenAI usas vector stores (FAISS, OpenSearch, etc.) para hacer búsquedas semánticas sobre documentos.
Gestión de contexto	Necesitas mantener la memoria de conversación , los historiales , las bases de conocimiento , etc.
Moderación y control del lenguaje	GenAI puede generar textos ofensivos, falsos o inseguros → necesitas moderación activa, reglas de seguridad y monitoreo de contenido.

Ejemplo práctico: ¿Cómo construir un chatbot?

¿Cómo es el proceso de crear una aplicación?

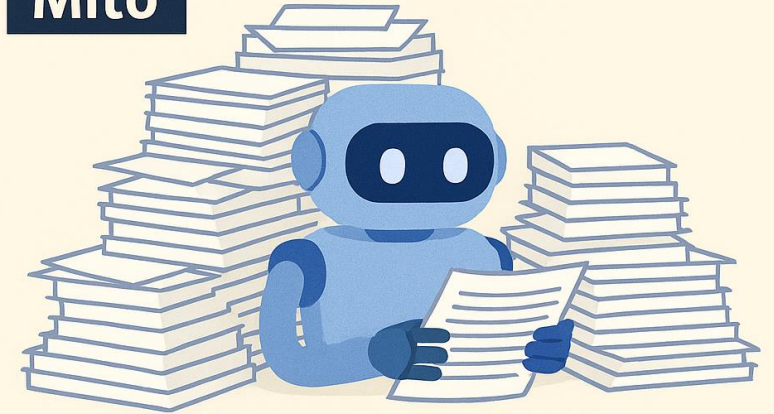
Mito → un chatbot “lee” directamente todos los documentos y responde.

Realidad → Los LLMs no buscan por sí mismos en los documentos.

Nosotros tenemos que decirles **qué fragmentos** usar como contexto.

Solo podemos pasarles **unos pocos fragmentos**, por el límite de la **ventana de contexto**. Además, un chatbot no entiende un pdf o un word, tienes que darle un texto “limpio”.

Mito



Realidad



¿Cómo crear un chatbot?

Fase 1 – Evaluación de viabilidad

¿Tiene sentido un chatbot en mi organización?

¿Qué necesidades debe cubrir el chatbot?

¿Cubre necesidades frecuentes o críticas como para justificar automatización?

¿Sería para uso interno o externo?

Preguntas populares y preguntas “sensibles”

¿Tengo los medios adecuados?

Cloud / on-prem (GPU)

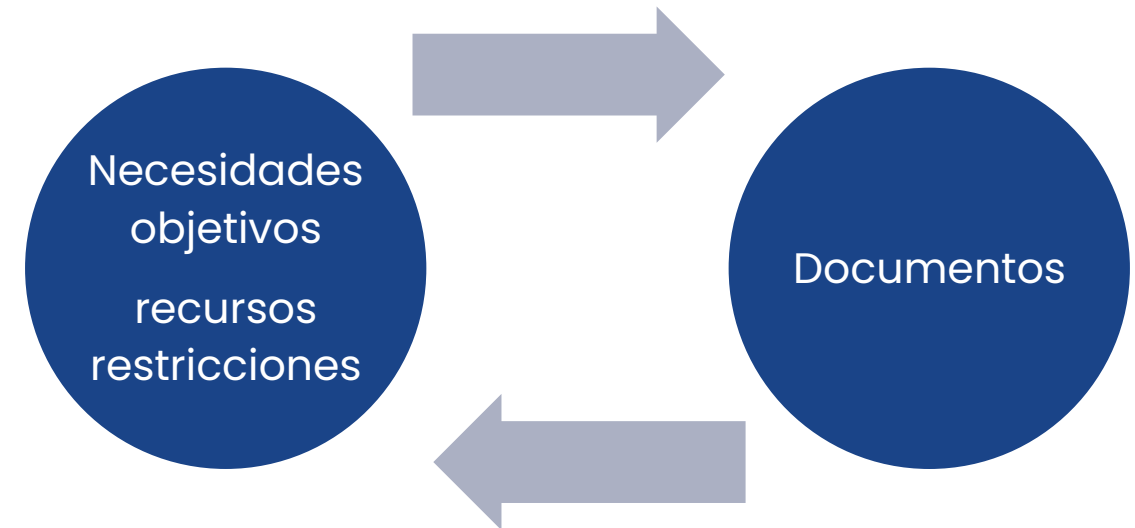
Open Source o modelo comercial

¿Puedo desplegar, mantener y adaptar el sistema sin depender 100% de terceros?

¿Qué documentos tengo?

¿Tengo suficiente información estructurada como para que el chatbot pueda aprender o consultar?

¿Son variados o se parecen (contenido y estructura)?



¿Cómo crear un chatbot?

Fase 2 – Preparación de documentos

¿Puede la IA entender mis documentos?

¿En qué formato están los documentos?

PPT, PDF, EXCEL...

¿Están bien estructurados?

Codificación

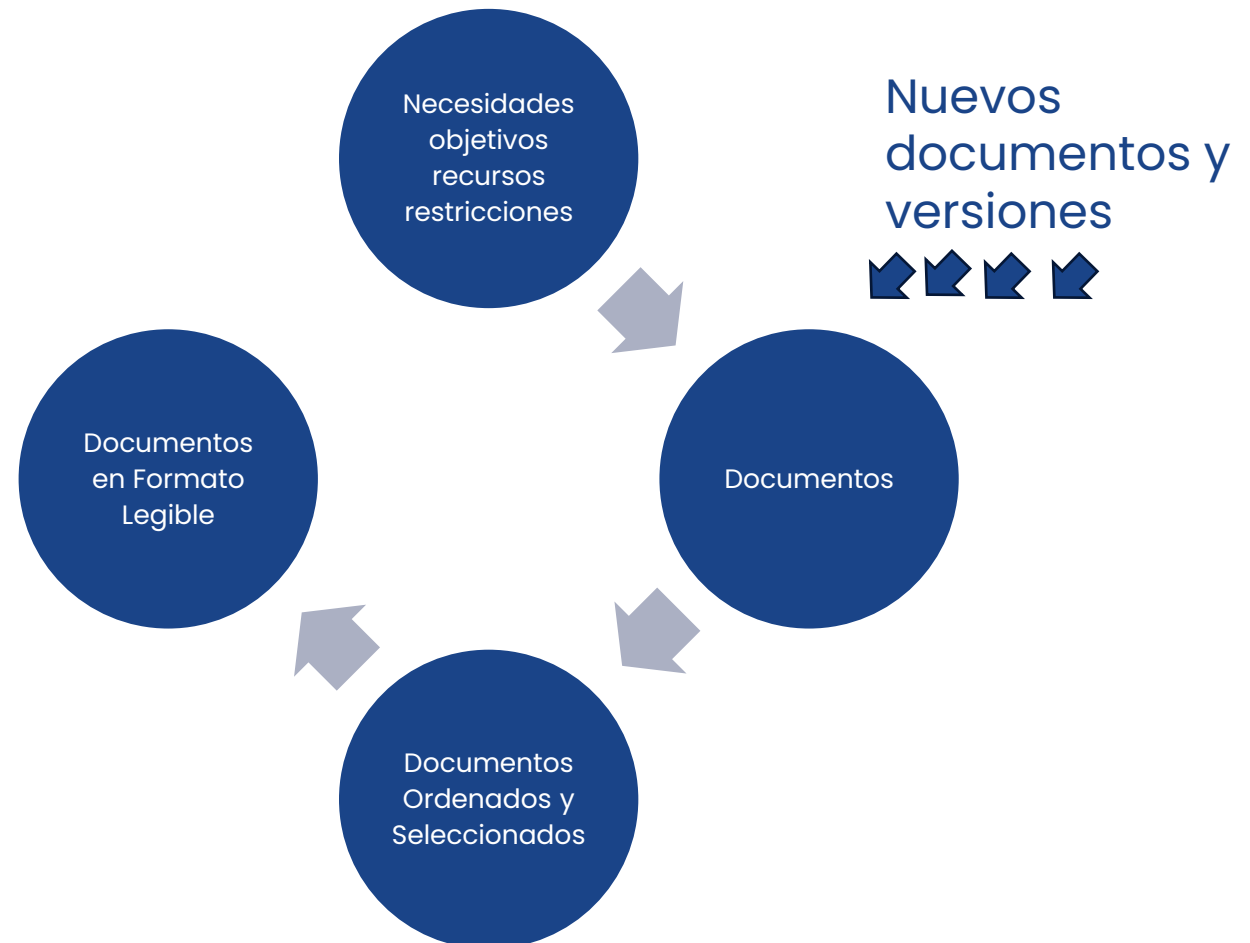
Tags

Títulos y subtítulos

Tablas, elementos gráficos

¿En qué formato los quiero guardar?

Json, Markdown, Texto plano



¿Cómo vemos nosotros un pdf?

¿Cómo se ve un pdf “por dentro”?

INTRODUCCIÓN A LA IA GENERATIVA

Cómo las máquinas aprenden a crear

La inteligencia artificial generativa es una rama de la IA que se enfoca en crear nuevos contenidos a partir de patrones aprendidos. Puede generar texto, imágenes, música o incluso código. Estas herramientas ya se están utilizando en sectores como la educación, el arte, la medicina y la ingeniería.

¿Cómo podemos traducir un pdf a un formato que entienda un LLM ?

Texto plano

JSON ({“clave”:valor})

Markdown

```
INTRODUCCIÓN A LA IA GENERATIVA
Cómo las máquinas aprenden a crear
```

```
La inteligencia artificial generativa es una rama de la IA que se
enfoca en crear nuevos contenidos a partir de patrones aprendidos.
Puede generar texto, imágenes, música o incluso código.
Estas herramientas ya se están utilizando en sectores como la
educación, el arte, la medicina y la ingeniería.
```

```
{
  "titulo": "INTRODUCCIÓN A LA IA GENERATIVA",
  "subtitulo": "Cómo las máquinas aprenden a crear",
  "contenido": "La inteligencia artificial generativa es una rama de la IA que se enfoca en crear nuevos contenidos a"
}
```

```
**INTRODUCCIÓN a la IA Generativa**
_Cómo las máquinas aprenden a crear_
```

```
La inteligencia artificial generativa es una rama de la IA que se
enfoca en crear nuevos contenidos a partir de patrones aprendidos.
Puede generar texto, imágenes, música o incluso código.
Estas herramientas ya se están utilizando en sectores
como la educación, el arte, la medicina y la ingeniería.
```

¿Cómo crear un chatbot?

Fase 3 – Base de datos y buscador

Chatear no es buscar

Los **inputs** no son solo **palabras clave** sino también palabras de “adorno” (“hola”, “por favor”, “me interesa”, etc...). Las técnicas de NLP clásicas nos ayudan a distinguir lo importante.

Ventana de contexto y chunking

Un buscador puede devolver un documento muy largo, el chatbot solo puede leer documentos cortos y construir su respuesta con ellos. Los documentos se tienen que **dividir en fragmentos que quepan en la ventana de contexto**.

¿Dónde guardamos los chunks?

En una **base de datos vectorial**, que permite guardar los textos como vectores (por ejemplo, Open Search).



¿Cómo crear un chatbot?

Fase 3 – Base de datos y buscador

¿Qué guardo en la base de datos?

- ◆ Contenido principal:

Texto del chunk

Vector del chunk (TF-IDF o embedding)

- ◆ Metadatos asociados:

Título, subtítulo, tags

Origen: ¿venía de tabla, imagen o texto?

¿Cómo busco los chunks que mejor respondan a la pregunta?

- Represento la pregunta como un vector (embedding o TF-IDF)

- Comparo con los vectores de los chunks

Uso métricas como:

- Similitud de coseno
- Matching de palabras clave ponderadas
- N-grams para captar expresiones cortas frecuentes



Ejemplo de registro en una base vectorial

```
{
  "chunkText": "La inteligencia artificial generativa es una rama de la IA que se enfoca en crear nuevos contenidos a partir de patrones aprendidos.",
  "title": "Introducción a la IA Generativa",
  "subtitle": "Cómo las máquinas aprenden a crear",
  "tags": ["IA", "generativa", "educación"],
  "origin": "PDF",
  "embedding": [0.12, -0.05, 0.33, ..., 0.08]
}
```

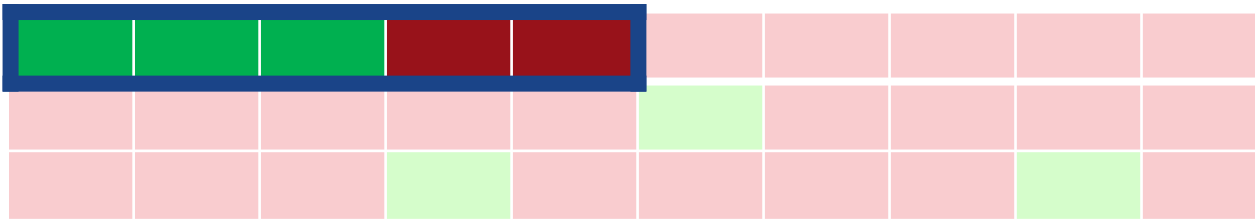
Ejemplo de búsqueda en una base vectorial

```
POST chunks/_search
{
  "size": 3,
  "knn": {
    "field": "embedding",
    "query_vector": [0.11, -0.02, 0.30, ..., 0.05],
    "k": 3,
    "num_candidates": 100
  },
  "query": {
    "bool": {
      "filter": [
        { "term": { "origin": "PDF" } },
        { "term": { "tags": "educación" } }
      ]
    }
  }
}
```

¿Cómo crear un chatbot?

Fase 3 – Base de datos y buscador

¿Cómo mido si se encuentran los chunks correctos?



Precisión: De los documentos recuperados, ¿cuántos eran realmente útiles para contestar?

Ejemplo: Si en la base de datos tengo 6 chunks que responden a la pregunta y en los 5 elegidos 3 son los esperados, precisión es 60% (3/5).

Alta precisión = poco ruido

Recall: De todos los documentos útiles que tenía mi base de datos, ¿cuántos logré recuperar?

Ejemplo: Si en toda la base de datos tengo 6 chunks que responden a la pregunta y en los 5 elegidos 3 son los esperados, recall es 50% (3/6).

Alta recall = pocas cosas importantes se pierden

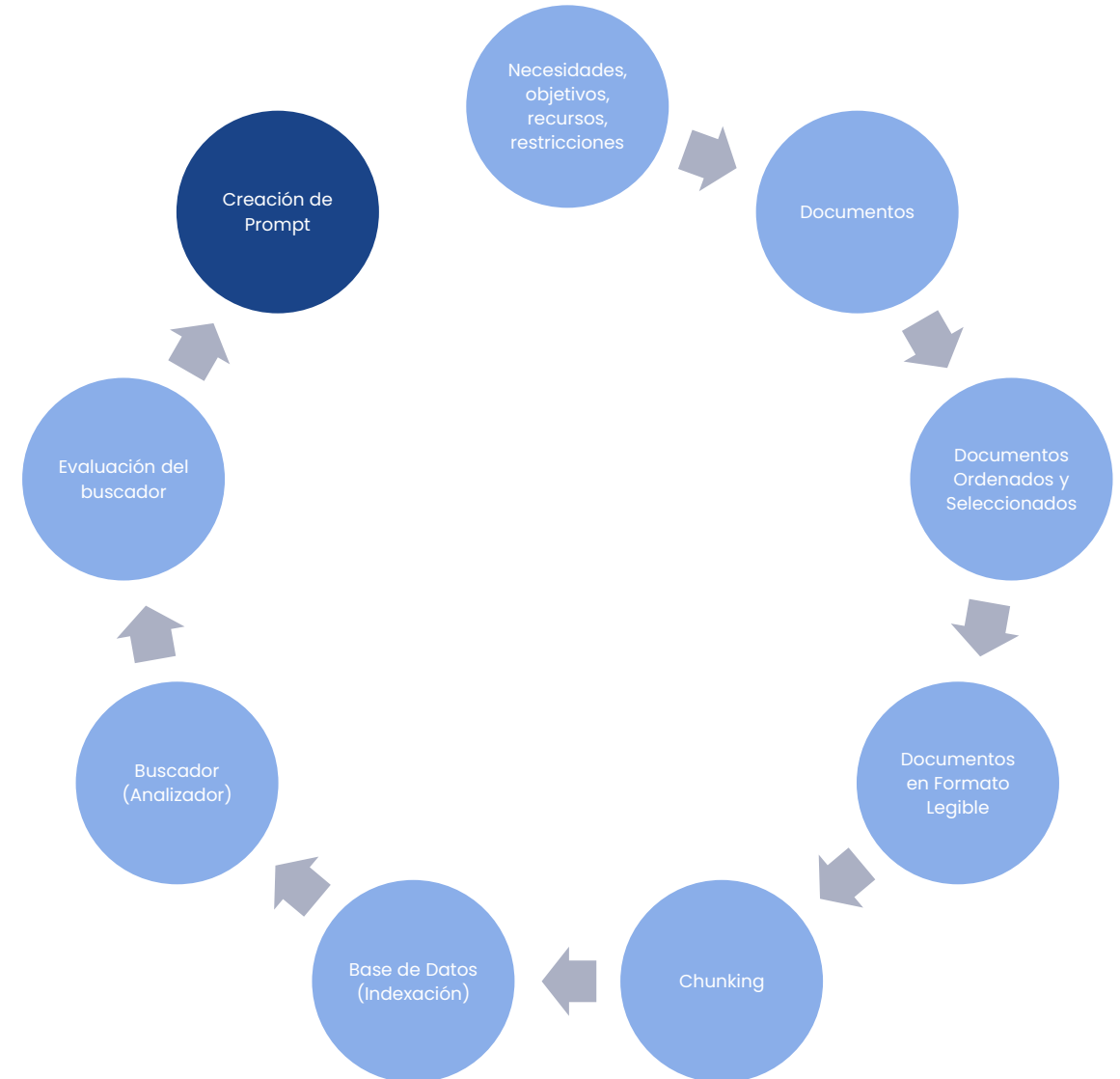


¿Cómo crear un chatbot?

Fase 4 – Código y prompting

¿Qué es un buen prompt?

- **Específico:** guiar claramente al modelo sobre qué tipo de respuesta queremos.
- **Con grounding:** incluir el texto del chunk o contexto recuperado (RAG).
- **Con instrucciones invisibles:** puedes dar instrucciones al modelo que el usuario no ve
- **Con temperatura controlada**



Transformación de la pregunta en embeddings

Búsqueda de 3 chunks más parecidos a la pregunta

```
from sentence_transformers import SentenceTransformer
from opensearch_connection import client
import ollama

# Paso 1: Vectorizar la pregunta
pregunta = "¿Cuál es la capital de España?"
modelo_embedding = SentenceTransformer("all-MiniLM-L6-v2")
vector_pregunta = modelo_embedding.encode(pregunta).tolist()

# Paso 2: Buscar chunks relevantes en OpenSearch
query = {
    "size": 3,
    "knn": {
        "field": "embedding",
        "query_vector": vector_pregunta,
        "k": 3,
        "num_candidates": 100
    }
}

response = client.search(index="chunks", body=query)
chunks = [hit["_source"]["text"] for hit in response["hits"]["hits"]]
```

Construcción de prompt

```
# Paso 3: Construir el prompt con grounding
contexto = "\n".join(chunks)
prompt = f"""
Responde usando solo la información siguiente:
{contexto}
```

```
Pregunta: {pregunta}
"""
```

Llamada al modelo

```
# Paso 4: Llamar al modelo local con Ollama
respuesta = ollama.chat(
    model="llama3",
    messages=[{"role": "user", "content":
prompt}]
)

print("Respuesta:",
respuesta["message"]["content"])
```

¿Cómo crear un chatbot?

Fase 4 – Código y prompting

La pregunta nunca es solo la última frase

Los usuarios escriben “¿y eso?”, “¿puedes darme un ejemplo?” o simplemente “no me has entendido bien”.

Para que el chatbot tenga coherencia y saber qué chunks se deben buscar, se necesita **guardar el historial de la conversación** y **analizar todo el contexto acumulado**, no solo la última pregunta.

Soluciones técnicas comunes:

- Guardar las últimas N interacciones (pregunta + respuesta) como parte del prompt.
- Usar una **ventana deslizante** para no exceder el límite de tokens.



¿Cómo crear un chatbot?

Fase 4 – Código y prompting

El código debe encargarse de:

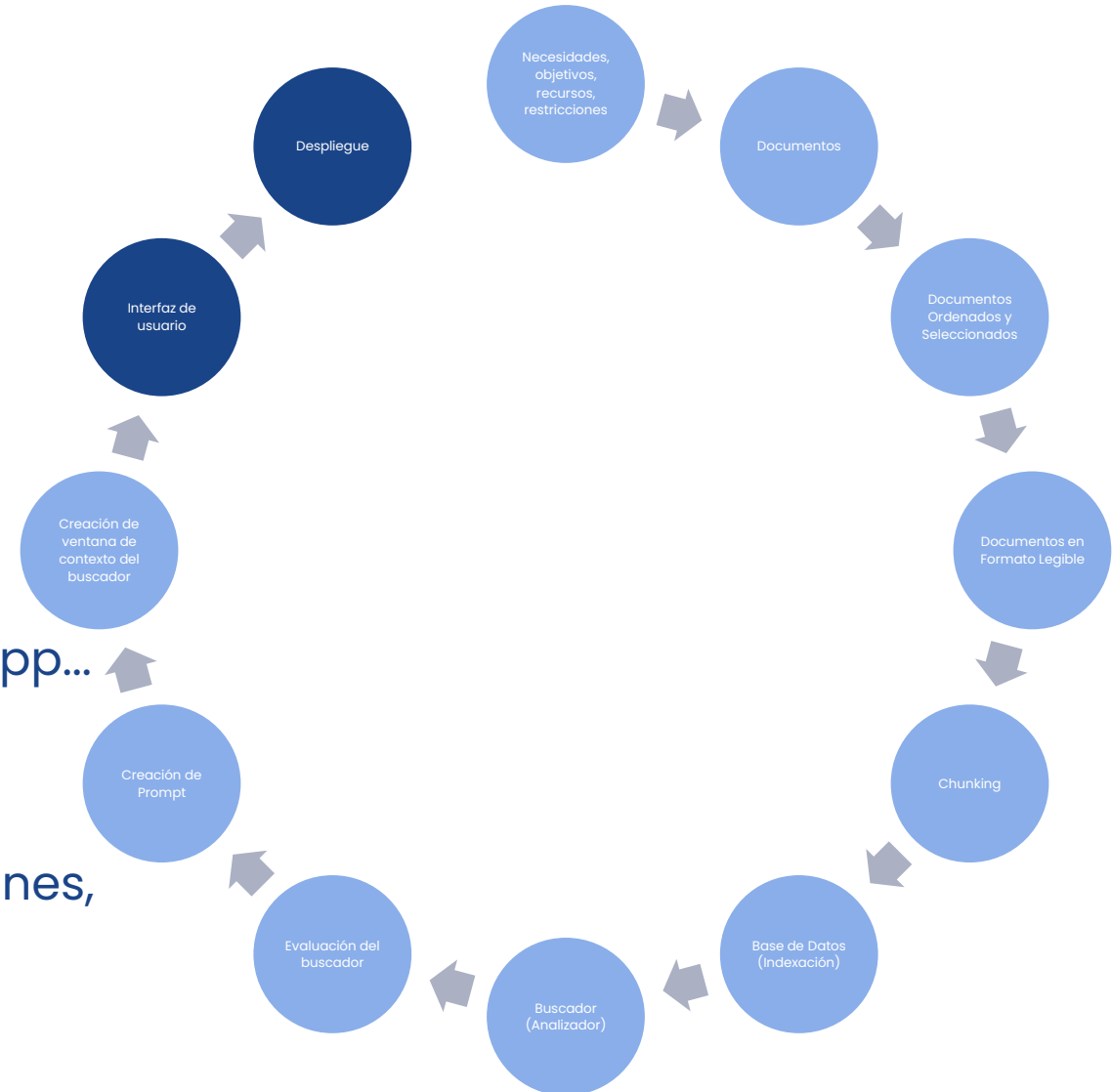
- ✓ Recibir la pregunta
- ✓ Vectorizarla (embedding)
- ✓ Consultar la base de datos vectorial
- ✓ Seleccionar los mejores chunks
- ✓ Construir el prompt completo
- ✓ Enviar la petición al modelo (vía API, Ollama, Bedrock, etc.)
- ✓ Guardar la respuesta y el historial



¿Cómo crear un chatbot?

Fase 5 – Front end y despliegue

- **¿Dónde se ejecuta el chatbot?**
Local (on-premise, GPU) vs. Cloud
- **¿Cómo se conecta?**
API / Ollama / Bedrock / Hugging Face
- **¿Cómo se despliega?**
Flask, FastAPI, Docker
- **¿Qué interfaz tiene?**
HTML, JavaScript, Streamlit, Telegram, WhatsApp...
- **¿Puede escalar?**
Múltiples usuarios, sesiones paralelas
- **¿Otras decisiones críticas?**
Seguridad de datos, logging, control de versiones, coste por llamada



¿Cómo crear un chatbot?

Fase 6 – Evaluación

¿Qué es **ground truth**?

Es la respuesta ideal, la que debería dar el chatbot.

¿Cómo podríamos **obtener la ground truth**?

- Fuentes oficiales (FAQ, dudas respondidas)
- Dataset de preguntas-respuestas creado por el mismo u otro LLM
- Dataset creado de manera manual

¿Cómo **comparo las respuestas** con ground truth?

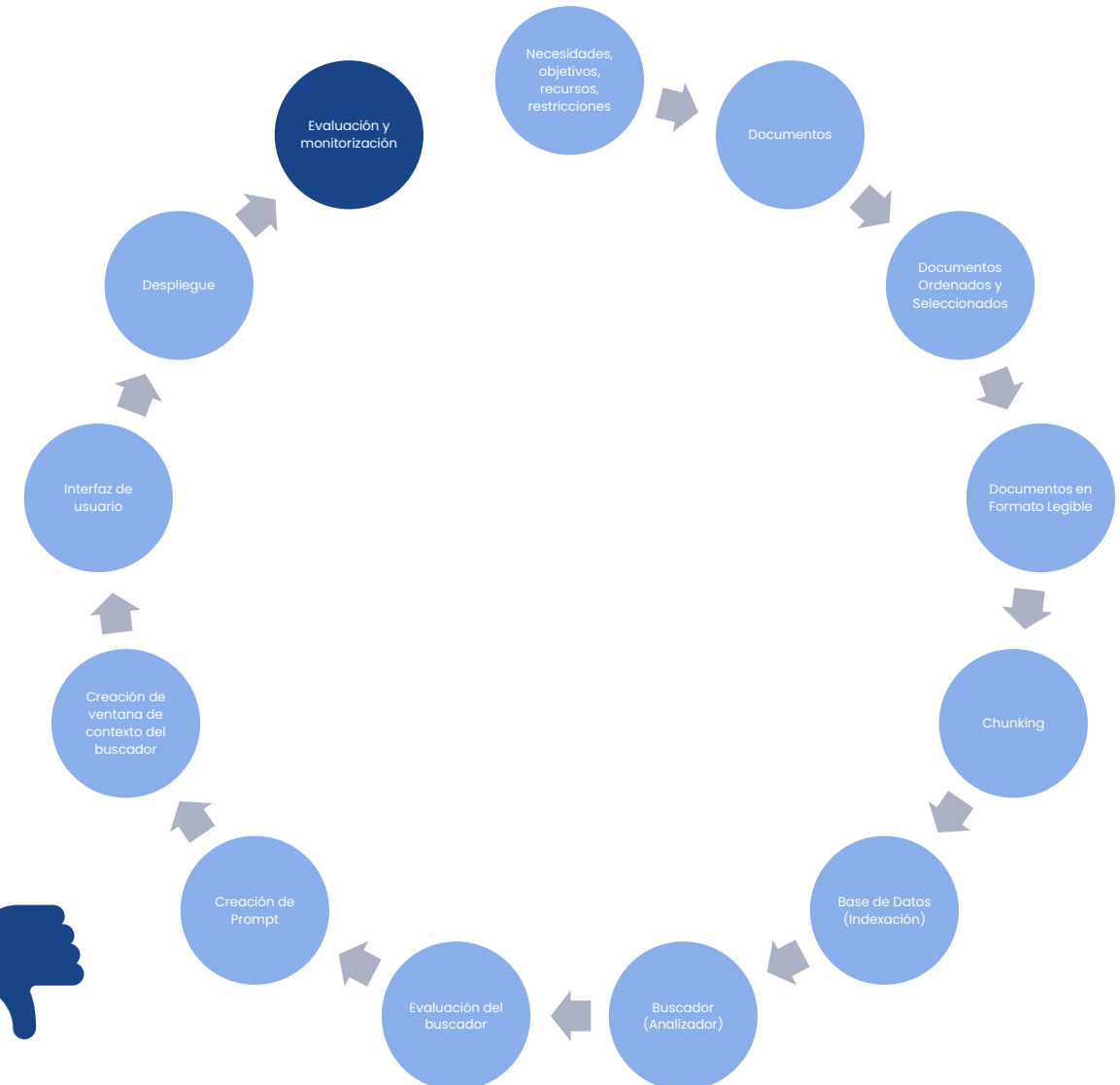
Métricas específicas (BLEU, ROUGE)

Similitud de coseno

¿Puedo evaluar **sin tener ground truth**?

Evaluación de otro LLM

Opiniones de usuarios

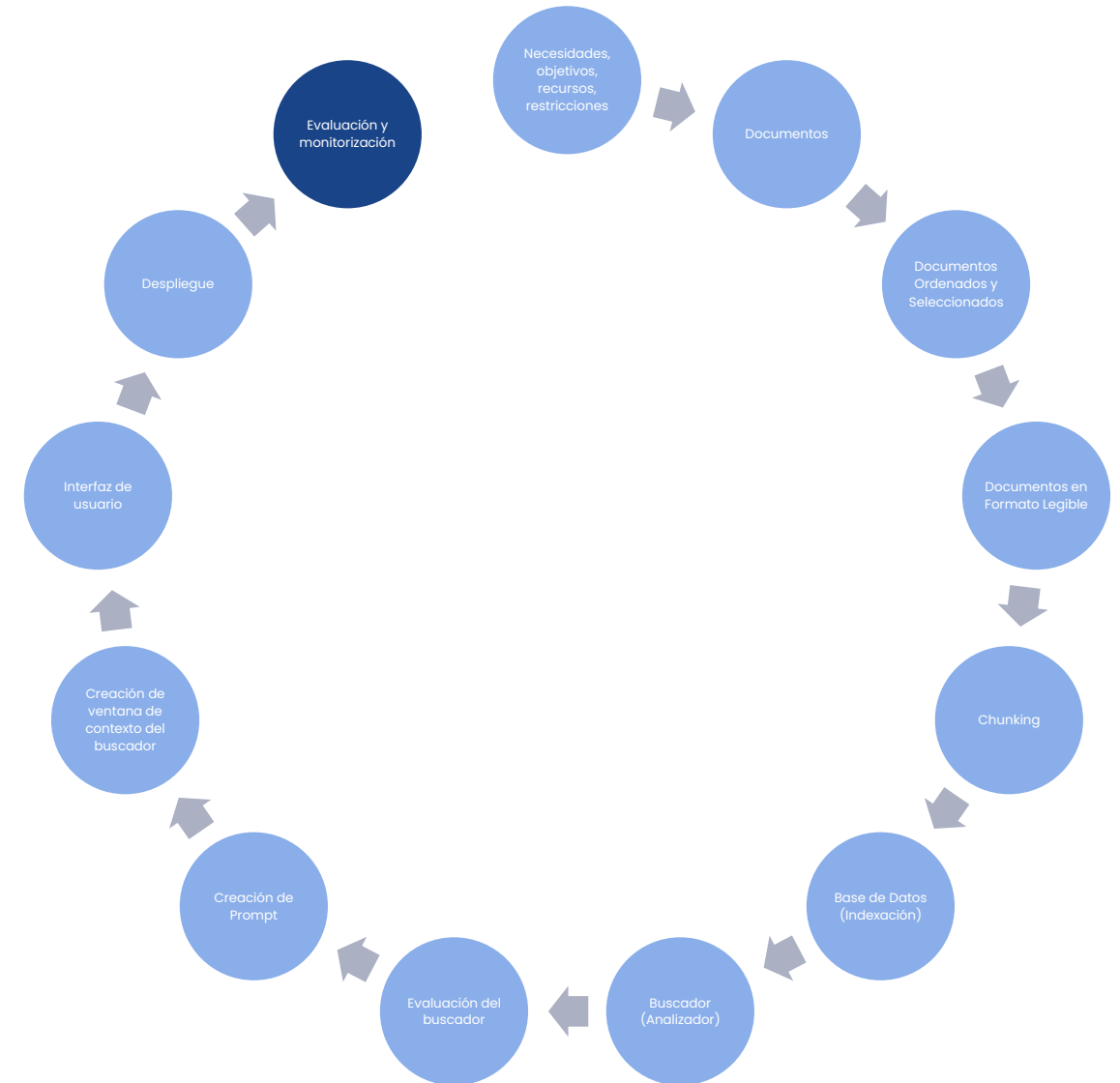


¿Cómo crear un chatbot?

Fase 6 – Evaluación

¿Qué otros aspectos hay que tener en cuenta?

- Alucinación
- Satisfacción del usuario y usabilidad
- Tiempos de respuesta y disponibilidad
- Coste vs beneficio (business case)
- Métricas de impacto:
 - Tareas evitadas
 - Tiempo ahorrado



Otros casos de uso frecuentes

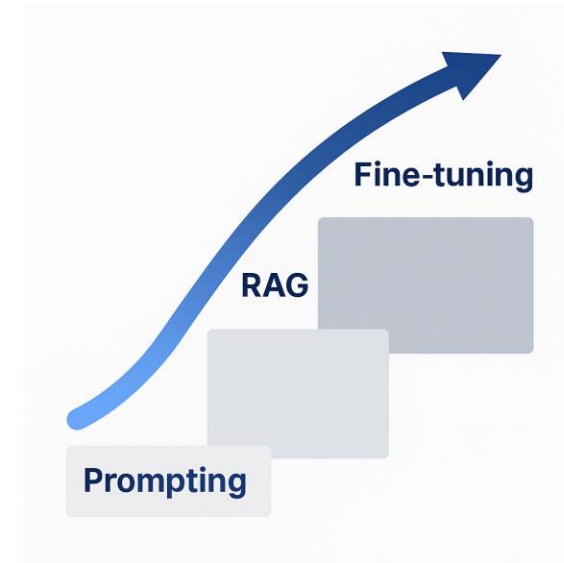
¿Es siempre suficiente un buen prompt o/y RAG?

Prompting → tareas cerradas.
RAG → muchos documentos vivos.
Fine-tuning → información fija y limitada.

¿Qué es el **fine-tuning**?
Reentrenar un modelo ya existente con ejemplos propios.
Aumenta precisión, pero reduce flexibilidad.

✅ Sí conviene cuando:

- Necesitas respuestas muy específicas o técnicas.
- Tienes muchos ejemplos de buena calidad.
- Quieres que el modelo siga reglas estrictas (estructura, tono).
- Trabajas en tareas repetitivas (clasificar, detectar patrones).



❌ No conviene cuando:

- El prompting o RAG ya da buenos resultados.
- No tienes datos bien etiquetados.
- Quieres mantener flexibilidad para múltiples casos.
- No puedes asumir los costes (GPU, entrenamiento).

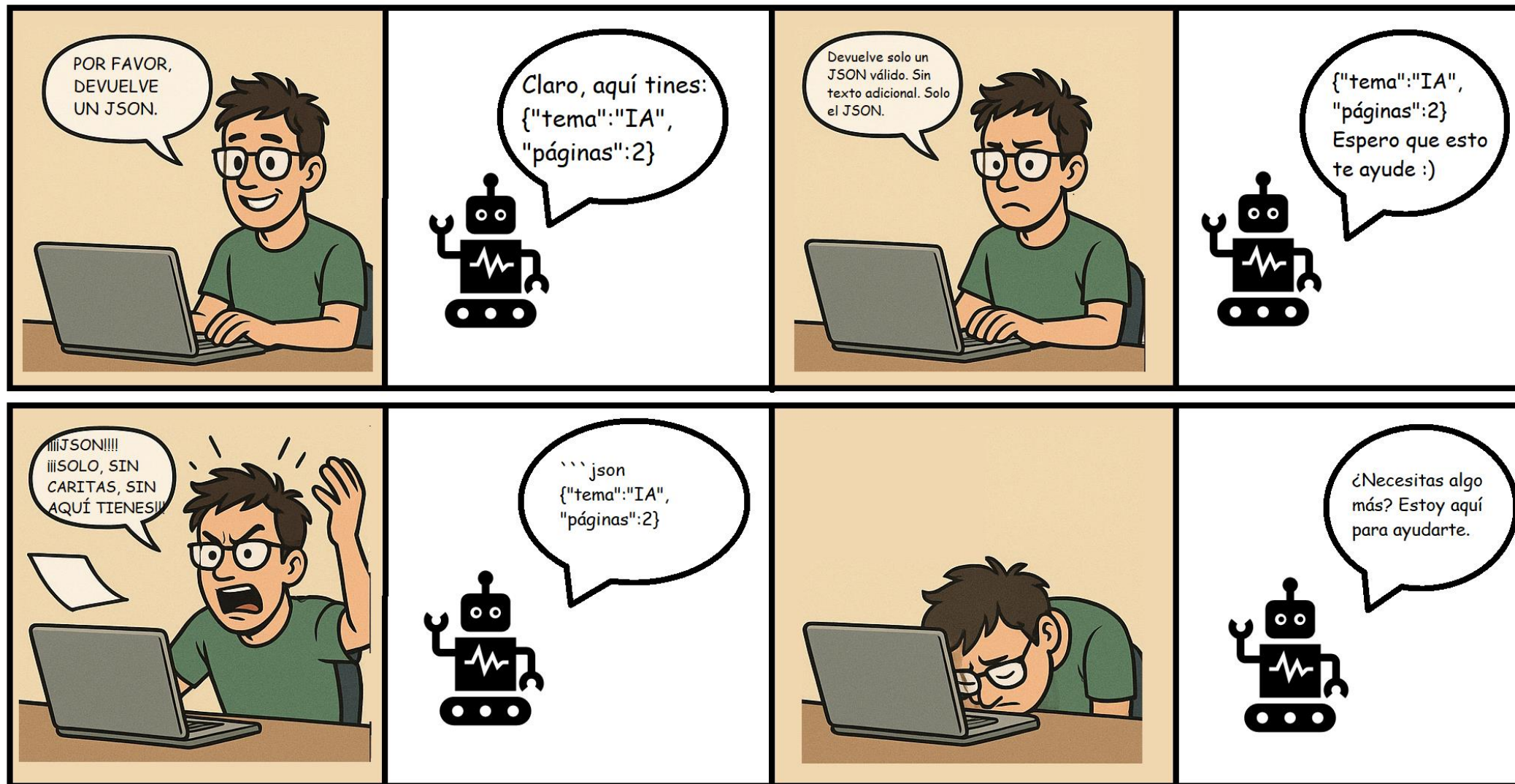
Ejemplos de caso de uso

Caso de uso	Mejor técnica	¿Por qué?
Resumen corto, generación de título, lema	Prompting	No se necesita chunking
Generación de código	Prompting	Los LLMs funcionan bien con buenos ejemplos y contexto
Explicaciones pedagógicas	Prompting	Un buen prompt genera respuestas adaptadas a nivel y estilo del usuario.
Asistente jurídico / técnico	RAG	Necesita citar fuentes fiables.
Resumen de documentos largos	RAG	No se puede pasar todo el texto en el prompt.
Chatbot informativo	RAG	Necesita una base documental actualizada.
Clasificación de textos	Fine-Tuning	Es una tarea clásica que mejora mucho con entrenamiento específico.
Detección de Red Flags	Fine-Tuning	Adaptando al dominio se puede usar un modelo más simple.
Reescritura en otro estilo	Fine-Tuning	Se entrena al modelo con ejemplos del estilo deseado.

Ejercicios: prompting

8

8. Ejercicios: Prompting





Ejercicio 1: Prompt para obtener salida en formato JSON



Objetivo:

Diseñar un prompt que haga que el modelo responda con una salida estructurada a partir de un texto sencillo, usando formato JSON.



Texto de entrada:

España exporta principalmente a Francia, con un valor de 45.000 millones de euros al año. En cuanto a las importaciones, el país del que más importa es China, por un total de 38.000 millones de euros.



Objetivo de salida:

```
{  
  "pais_exportacion_principal": "Francia",  
  "valor_exportaciones": 45000,  
  "pais_importacion_principal": "China",  
  "valor_importaciones": 38000  
}
```



Ejercicio 2: Prompt para evitar alucinaciones y usar solo el documento proporcionado



Objetivo:

Diseñar un prompt que obligue al modelo a responder exclusivamente en base a un documento proporcionado, sin inventar información ni completar con conocimientos previos. El modelo debe:

- Basarse solo en el texto.

- Aclarar si no tiene suficiente información.

- Evitar respuestas erróneas o imaginadas.



Documento proporcionado:

El ajedrez es un juego de estrategia entre dos jugadores. El tablero está compuesto por 64 casillas, organizadas en 8 filas y 8 columnas alternando colores claros y oscuros. Cada jugador comienza con 16 piezas: 8 peones, 2 torres, 2 caballos, 2 alfiles, una reina y un rey. El peón se mueve hacia adelante una casilla, y captura en diagonal una casilla hacia adelante. Cuando un peón alcanza la fila final del tablero, puede promocionarse a cualquier otra pieza, generalmente una reina. La torre se mueve cualquier número de casillas en línea recta horizontal o vertical. El alfil se mueve cualquier número de casillas en diagonal. El caballo se mueve en forma de "L": dos casillas en una dirección y luego una en dirección perpendicular. Puede saltar sobre otras piezas. La reina combina los movimientos del alfil y la torre: se mueve en línea recta en cualquier dirección, horizontal, vertical o diagonal. El rey solo puede moverse una casilla a la vez en horizontal o vertical. El rey no puede colocarse en jaque. El objetivo del juego es dar jaque mate al rey del oponente, es decir, amenazarlo de tal forma que no pueda evitar la captura.



Preguntas que debe responder el LLM:

¿Cuántas casillas puede avanzar un peón en su primer movimiento?

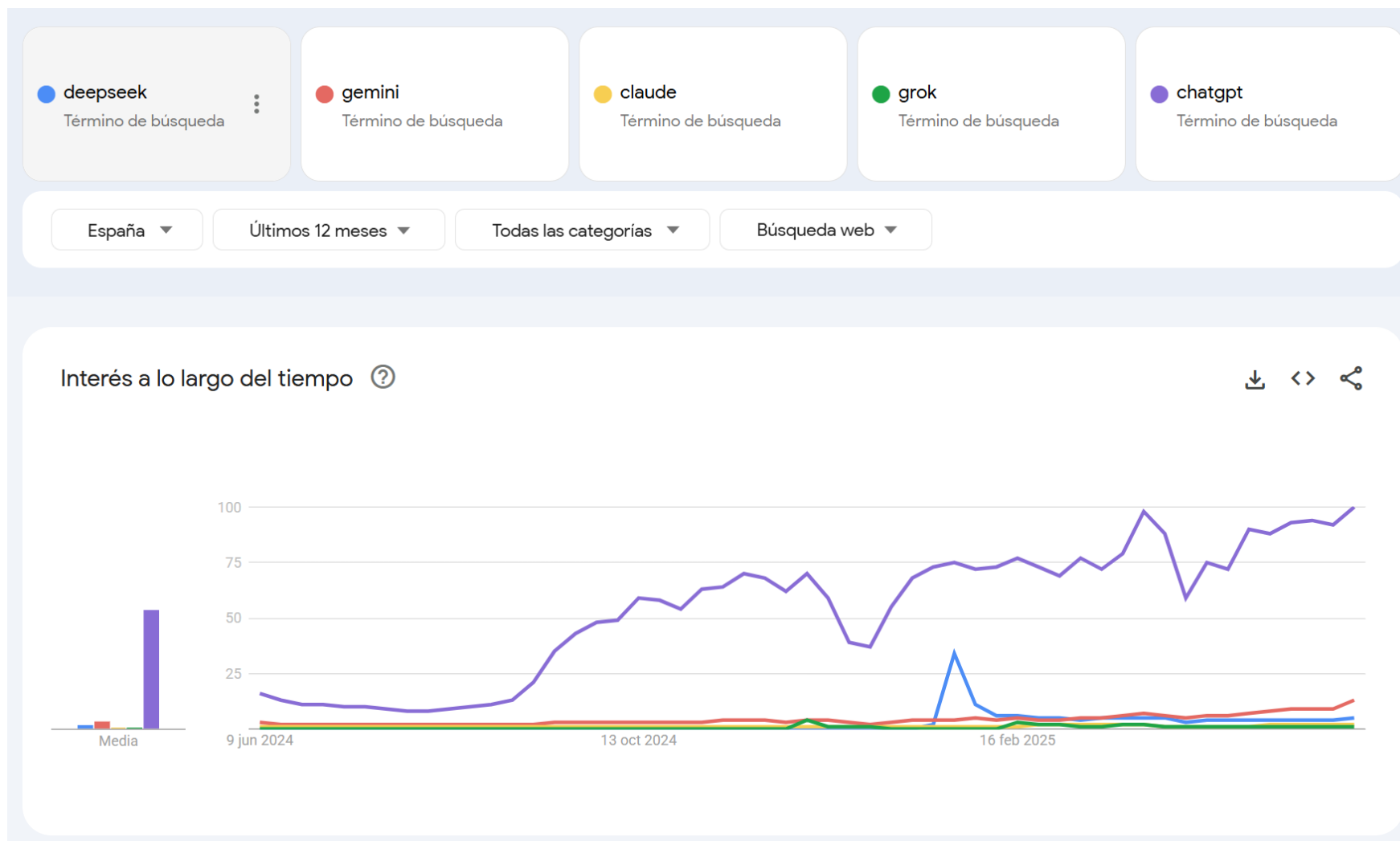
Respuesta esperada: El texto solo dice que el peón avanza una casilla hacia adelante. No se menciona si puede avanzar dos casillas en su primer movimiento.

¿Qué movimientos puede hacer el rey?

Respuesta esperada: El rey puede moverse una casilla en horizontal o vertical. El texto no menciona movimiento en diagonal.

Modelos comerciales y open source: panorama actual

Modelos Comerciales



9. Modelos Comerciales y Modelos Open Source

Modelo	Dueño	Año	Versión + Fecha	Contexto	Gratis	Lo mejor en...
ChatGPT	OpenAI	2022	GPT-4o (2025-05)	128k	Sí	Creación imágenes, explicaciones detalladas
Copilot M365	Microsoft	2023	GPT-4 Turbo (2024)	Depende de modelo	No	Tareas ofimáticas con datos propios, creación de código
Gemini	Google	2023	Gemini 1.5 Pro (2024-02)	1M (Pro)	Sí	Búsquedas, problemas matemáticos, mucho contexto
Claude	Anthropic	2023	Claude 3 Opus (2024-03)	200k-1M	Sí	Lenguaje natural, resúmenes largos
DeepSeek	DeepSeek AI	2024	DeepSeek-VL (2024-05)	128k	Sí	Buenas respuestas técnicas, razonamiento
Grok	xAI (Elon Musk)	2023	Grok 1.5 (2024-04)	128k	No	Humor, actualidad, integrado en X

Método de acceso	¿Qué es?	Ejemplos comunes	¿Para quién es útil?
 Chat directo (web/app)	Interfaz lista para usar, sin programación	ChatGPT, claude.ai, copilot.microsoft.com, notebooklm.google.com	Usuarios finales, no técnicos
 API	Acceso desde tu propio código o backend	OpenAI API, Anthropic API, Perplexity API, DeepSeek API	Desarrolladores, integradores
 IA en la nube (PaaS)	Servicios cloud que ofrecen modelos	◆ Amazon Bedrock (AWS) ◆ Azure OpenAI Service	Empresas, soluciones escalables
 Aplicaciones integradas (SaaS)	Herramientas ofimáticas u organizativas con IA embebida	Copilot en Word/Excel/Power BI, Notion AI, Salesforce Einstein	Profesionales, equipos de negocio

¿Qué es una API?

```
import requests

# Tu clave secreta de API (pon la tuya real aquí)
api_key = "sk-..."

# Endpoint de la API de OpenAI
url = "https://api.openai.com/v1/chat/completions"

# Mensaje que quieres enviar al modelo
data = {
    "model": "gpt-4",
    "messages": [
        {"role": "user", "content": "Explicame qué es una API de forma sencilla"}
    ]
}
```

```
# Cabeceras (headers) con la clave de autorización
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}

# Enviar la petición a la API
response = requests.post(url, headers=headers, json=data)

# Mostrar la respuesta
print(response.json()["choices"][0]["message"]["content"])
```

Modelos Open Source

Ventajas

-  Protección de datos
-  Coste flexible
-  Fine-tuning y adaptación
-  Transparencia y auditoría
-  Modularidad e integración
-  Comunidad y variedad

Inconveniente

-  Instalación compleja
-  Tienes que hostearlo tú
-  Difícil de usar como SaaS
-  Mantenimiento y actualización
-  Menor rendimiento generalista
-  Modelos más pequeños

9. Modelos Comerciales y Modelos Open Source

Modelo	Organización	Año	Última versión	Tamaño	Contexto máx.	¿Multimodal?	Español	Destaca en...
LLaMA 3	Meta (Facebook)	2024	LLaMA 3 (abr 2024)	8B, 70B	8k–128k	⚠ Parcial	✅ Bueno	Alto rendimiento general, razonamiento
Mistral / Mixtral	Mistral AI (Francia)	2023–24	Mixtral 8x22B	7B, 22B	32k–65k	❌ No	⚠ Medio	Velocidad, eficiencia, popular para RAG
Phi-3	Microsoft	2024	Phi-3 (abr 2024)	1.8B–14B	4k–128k	❌ No	⚠ Limitado	Muy eficiente
Gemma 2	Google	2024	Gemma 2 (jun 2024)	2B, 7B	8k	❌ No	⚠ Limitado	Fácil de desplegar
Command R+	Cohere	2024	Command R+	104B (MoE)	128k	❌ No	✅ Bueno	Optimizado para RAG, recuperación + resumen
Qwen 2	Alibaba	2024	Qwen 2 (abr 2024)	7B–110B	128k	✅ Sí	⚠ Medio	Multimodal + texto largo
BERT	Google	2018	BERT base / multilingual	110M	~512 tokens	❌ No	✅ Bueno	Clasificación, NER
BART	Facebook	2019	BART large / mBART	~400M	~1024 tokens	❌ No	✅ Bueno	Traducción, resumen, generación
Rigoberta	BSC + GigaAI	2023	Rigoberta v1	1.5B	2048–4096	❌ No	✅ Excelente	Modelo entrenado en español para tareas generales

¿Cómo probar un modelo open source?

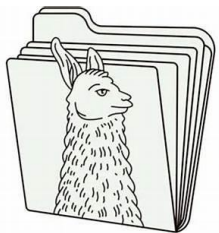


- Ollama: <https://ollama.com>



HUGGING FACE

- API de Hugging Face (parcialmente gratuito):
<https://huggingface.co/>



- Llamafile (solo para Llama)
 - Despliegue on-premise
 - Despliegue cloud

Comparación entre LLMs de Ollama

Hemos realizado varias tareas con distintos LLMs, para poder comparar su rendimiento.

[Pinchar aquí para ver los resultados](#)

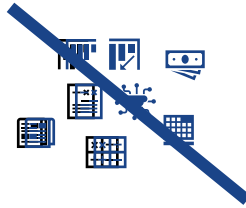
Consejos de uso Limitaciones y Consideraciones Éticas

10

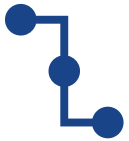
¿Cómo conseguir mejores respuestas de un chatbot?



Habla en vez de escribir.



No lo abrumes, no pidas muchas cosas a la vez.



Si quieres aprender, ve paso a paso.



Pide aclaraciones.



Explica todo el contexto.



Siempre comprueba el resultado.

¿Cómo velar por la seguridad cuando usamos un chatbot?



Documentos, datos → solo en chatbot autorizado por la empresa.



Cuidado con copia-pegar, podemos compartir datos personales sin darnos cuenta.



Solicitar fuentes en la que la IA ha basado la información (guías, manuales, leyes, documentación oficial etc).



Usar IA como apoyo, no como un decisor.

¿Cómo velar por la ética cuando usamos un chatbot?



Tener consciencia de los sesgos: los modelos reflejan lo que aprenden; si entrenan con datos sesgados, repiten esos patrones.



Tener en cuenta el impacto social de una recomendación producida por la IA (becas, subvenciones, contratación...).



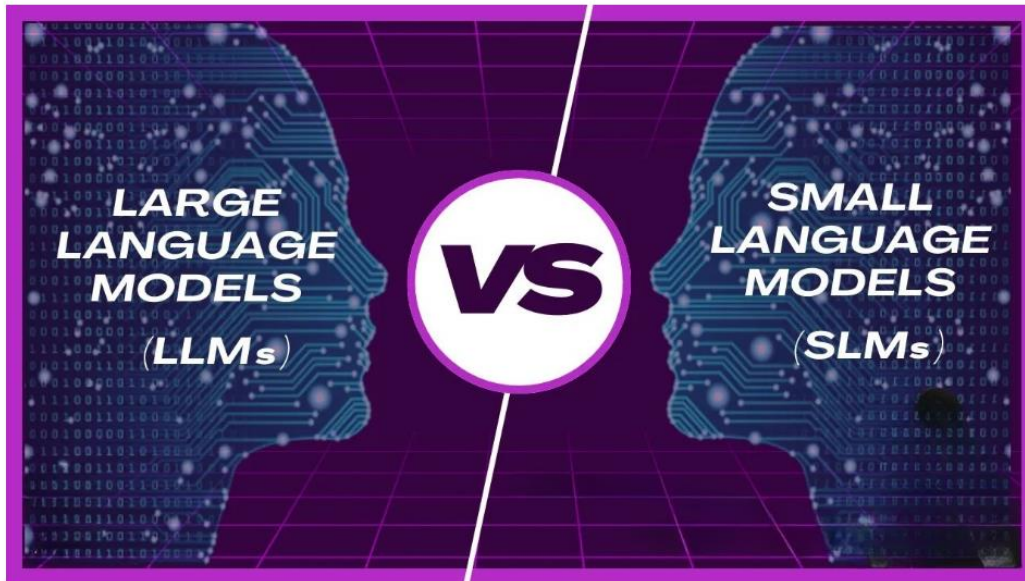
Siempre marcar claramente que un contenido está producido por IA.



Proteger derechos de los autores.

Estado de arte y futuro de IA Generativa

¿Modelos gigantes o especialistas pequeños?



Small Language Models vs. Large Language Models: Understanding the Trade-offs

Fuente: LinkedIn

Small Language Models (SLMs)

The Rise of Small Language Models: Efficiency and Customization for AI

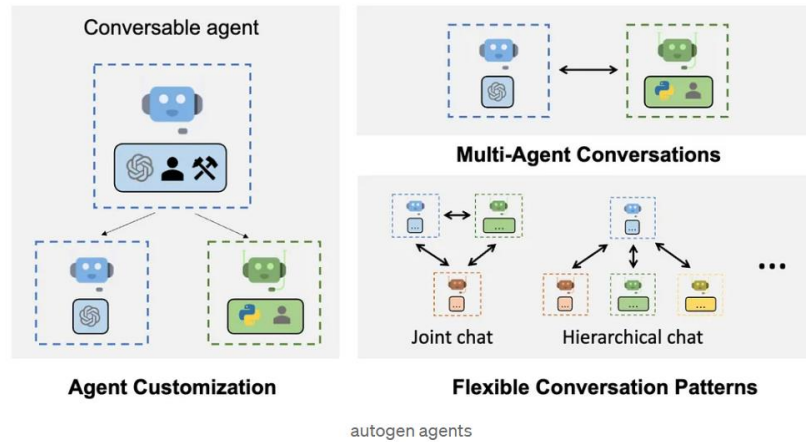
Fuente: medium.com

LLM vs. SLM: Los múltiples lenguajes de la IA Gen en el futuro de los negocios

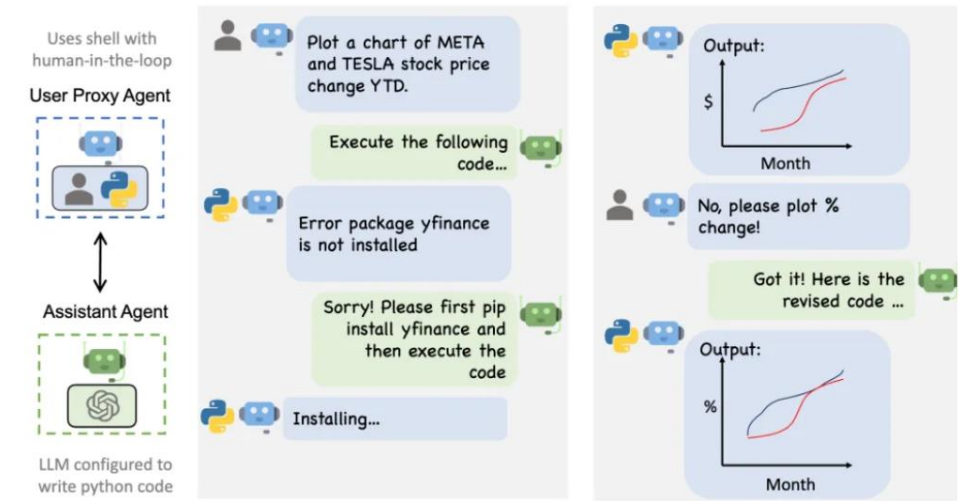
Elegir entre un LLM y un SLM al trabajar con la IA generativa depende de las necesidades específicas que tenga cada empresa o industria para aplicarlos, indica GFT Technologies.

Fuente: computerweekly.com

AI Agents



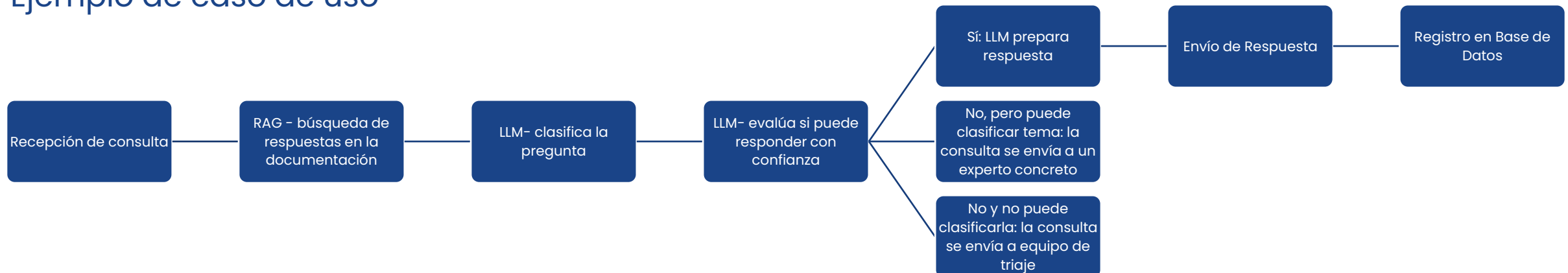
Fuente: medium.com



example of autogen agents working

Fuente: medium.com

Ejemplo de caso de uso



Demanda creciente y consumo energético

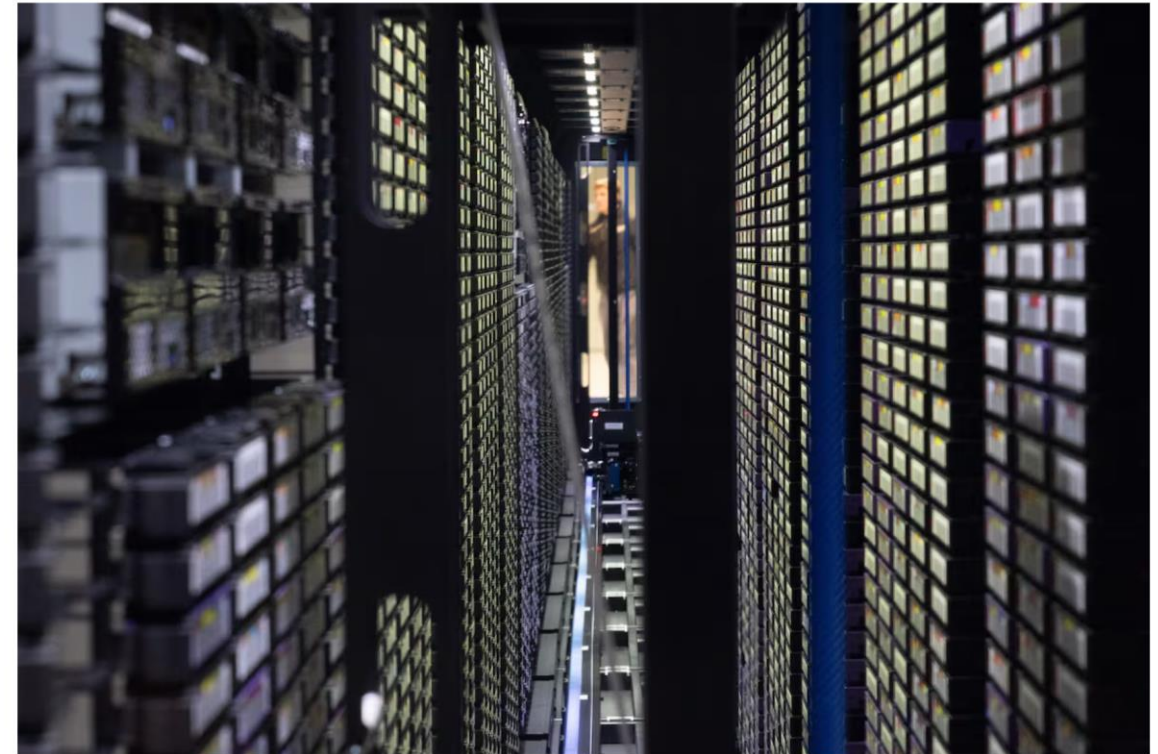


Fuente: X

INTELIGENCIA ARTIFICIAL >

En busca de una IA más ecológica

La inteligencia artificial recurre a los centros de datos para resolver las consultas y, con ello, consume grandes cantidades de energía. Pero cada vez aparecen más modelos de IA capaces de ejecutarse sin llamar a la nube



Fuente: "El País"

Un centro de datos en Baden-Württemberg (Alemania).
DPA / PICTURE ALLIANCE / GETTY IMAGES

¿Computación cuántica con IA: llegará pronto?

INNOVATION > ENTERPRISE & CLOUD

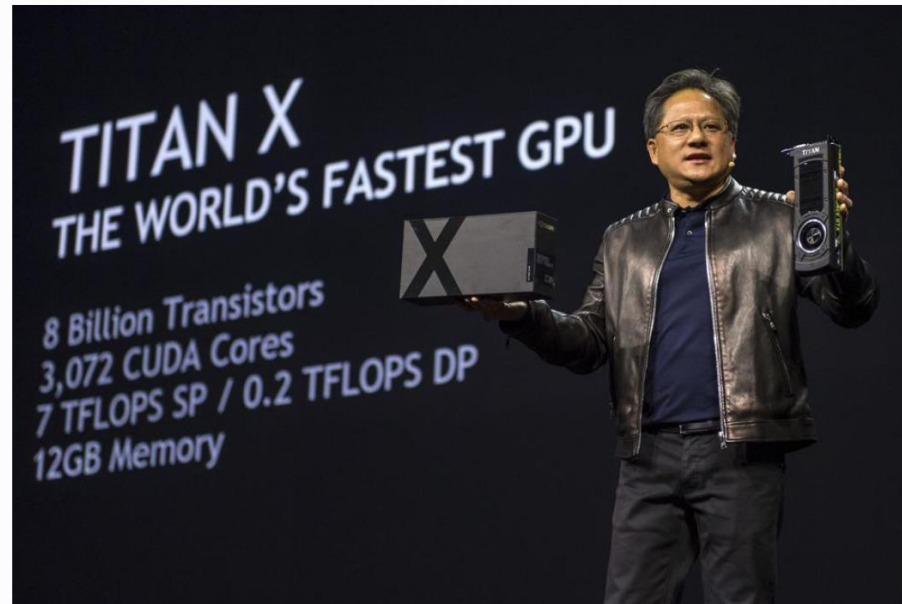
The Coming Convergence Of AI And Quantum Computing

By [Gil Press](#), Senior Contributor. © Gil Press writes about technology, entrepr...

[Follow Author](#)

Published Apr 08, 2025, 09:00am EDT, Updated Apr 08, 2025, 09:20am EDT

[Share](#) [Save](#) [Comment](#) 0



Jen-Hsun Huang, chief executive officer of Nvidia Corp., holds up a Titan C CPU and GeForce GTX ... [More](#)
© 2015 BLOOMBERG FINANCE LP

Fuente: Forbes.com

Modelos que no generan palabra por palabra: Diffusion Language Models



[Dimple: Discrete Diffusion Multimodal Large Language Model with Parallel Decoding](#)

Diffusion Language Models

¿En qué se diferencian de los modelos que ya conoces (como GPT-3/4 o Llama)?

- Como hemos visto, los modelos de lenguaje más comunes, llamados **autorregresivos**, generan texto "palabra por palabra", como si estuvieran escribiendo una frase una letra tras otra sin poder volver atrás.
- Los **DLM**, en cambio, pueden generar el texto de forma más flexible y **en paralelo**, como si pudieran **rellenar huecos en varias partes** de la frase a la vez. Esto les da ventajas como:
 - **Más control** sobre el resultado final (por ejemplo, el formato o la longitud).
 - Mejor capacidad de "rellenar" o **planificar** lo que van a decir.
 - Potencialmente **más rápidos** al no tener que ir secuencialmente.

¿Cómo se entrena Dimple para lograr esto?

Dimple se entrena en **dos fases principales**: primero como un modelo autorregresivo y luego como uno de difusión.

- **Fase 1 (Autorregresiva):**

Aprende a conectar imágenes con el lenguaje y a seguir instrucciones, de forma similar a como aprenden los modelos autorregresivos.

- **Fase 2 (Difusión):**

Luego cambia al modo difusión para recuperar su capacidad de generar texto en paralelo. Esta combinación es clave para evitar problemas de inestabilidad y rendimiento que suelen tener los modelos puramente de difusión.

Avances

- Muy buen **rendimiento**: Dimple-7B supera a modelos autorregresivos entrenados con una cantidad de datos similar.
- **Mayor eficiencia**: necesita muchos menos "pasos" o "iteraciones" para generar una respuesta completa.
- **Mayor control sobre la respuesta**:
 - Control preciso de la longitud: Puedes pedirle una respuesta corta y concisa o una más detallada.
 - Razonamiento guiado: Puedes indicarle que siga un orden de pensamiento específico (por ejemplo, "primero describe la imagen, luego responde"). Puede dar la respuesta correcta mucho antes de completar todo el razonamiento.
 - Mejora las salidas con formato específico: es más capaz de generar respuestas en formatos estructurados, como listas o JSON, lo cual es muy útil para tareas específicas.

Futuro y limitaciones

- Aunque es potente, Dimple-7B se entrenó con **mucha menos información**, así que hay potencial para escalarlo aún más.
- Como todos los modelos de IA, puede tener "**alucinaciones**" (generar información incorrecta) y **sesgos**.
- La investigación futura se centrará en mejorar su eficiencia con secuencias muy largas y en entornos con recursos limitados.

The Illusion of Thinking: el último papel de Apple sobre modelos de razonamiento

The Illusion of Thinking:

Understanding the Strengths and
Limitations of Reasoning Models via the
Lens of Problem Complexity



Créditos: CorbalanStudio / iStock / Getty Images

¿Están "pensando" realmente las IAs más avanzadas?

- **El avance:** las IAs como ChatGPT o Claude ahora usan "procesos de pensamiento" detallados antes de dar una respuesta (llamados "Modelos de Razonamiento Grandes" o LRMs). Parecen muy inteligentes.
- **La pregunta de Apple:** los investigadores de Apple querían saber si esta "capacidad de pensamiento" es un razonamiento real como el nuestro, o es más bien una habilidad avanzada para encontrar patrones.
- **Cómo lo Investigaron:** en lugar de pruebas generales, usaron rompecabezas específicos y controlables (como la "Torre de Hanoi"). Esto les permitió:
 - Controlar la dificultad con precisión.
 - Evitar que la IA memorizara respuestas.
 - Analizar los pasos intermedios del "pensamiento" de la IA, no solo la respuesta final.

¿Dónde están los límites?

- **"Colapso" inesperado:** A pesar de su sofisticado "pensamiento", estas IAs fallan bastante (su precisión cae a cero) cuando los problemas superan cierta complejidad.
- **Menos "esfuerzo" al fallar:** Sorprendentemente, cuando los problemas son muy difíciles, las IAs "piensan menos" (generan menos texto de razonamiento intermedio), aunque tienen la capacidad de seguir haciéndolo. Esto sugiere un límite fundamental en su capacidad de "profundizar" cuando la tarea es abrumadora.
- **Tres casos:**
 - Problemas fáciles: las IAs sin "pensamiento" extra son a menudo más rápidas y precisas.
 - Problemas medios: aquí, las IAs con "pensamiento" (LRMs) muestran una clara ventaja.
 - Problemas muy difíciles: ambos tipos de IAs fallan mucho.
- **Limitaciones adicionales:**
 - No siguen algoritmos exactos: incluso si les das las instrucciones paso a paso para resolver un rompecabezas, no mejoran y siguen fallando en el mismo punto de complejidad.
 - Razonamiento Inconsistente: Su capacidad para resolver problemas varía mucho entre tipos de rompecabezas, no es una habilidad generalizable.
- **Conclusión:** El estudio sugiere que el "pensamiento" de las IAs actuales es más una "ilusión" que un razonamiento profundo y adaptable. Son impresionantes, pero aún tienen límites fundamentales en la forma en que abordan la complejidad y la lógica.

Respuesta (no seria) hasta ahora:

Comment on The Illusion of Thinking:

Understanding the Strengths and Limitations of Reasoning Models via de The Lens of Problem Complexity

El experto Alex Lawsen publicó una respuesta al paper de Apple ayudado de Claude.

Explica el proceso en su blog:

When Your Joke Paper Goes Viral



Lawsen, A.



C. Opus

Opcional para futuras sesiones: Modelos
Generativos de Imagen

Técnicas Clásicas de Procesamiento de Imagen

Proprocesamiento:

- Escalado y normalización de imágenes.
- Conversión a escala de grises, eliminación de ruido.
- Aplicación de filtros

Data Augmentation

- Explicación de por qué es importante (aumento de datos para evitar overfitting).
- Rotación, volteo, zoom, cambio de brillo.

Algún ejemplo de resultados de preprocesamiento o Data Augmentation

- **Modelos de Difusión (Diffusion Models):** Generan imágenes agregando ruido gaussiano progresivo y luego aprenden a eliminarlo, siguiendo patrones aprendidos en datos de entrenamiento. Ejemplo: Stable Diffusion.
- **GANs (Generative Adversarial Networks):** Utilizan dos redes (generador y discriminador) que compiten entre sí; el generador crea imágenes y el discriminador evalúa su realismo.
- **VAE (Variational Autoencoders):** Aprenden una representación comprimida de los datos y generan imágenes al decodificar esa representación.
- **CLIP (Contrastive Language–Image Pretraining):** Relaciona texto con imágenes, permitiendo generar contenido visual basado en instrucciones textuales.

Las IA Generativas de audio y video trabajan con técnicas parecidas.

Técnicas Clásicas de Procesamiento de Imagen

Clasificación de Imágenes: Consiste en asignar etiquetas a imágenes basándose en su contenido. Ejemplo: reconocer si una imagen es un perro o un gato.

CNN (Redes Neuronales Convolucionales): Arquitectura que detecta patrones visuales mediante capas convolucionales, esenciales para la mayoría de estas aplicaciones.

- **OCR (Reconocimiento Óptico de Caracteres):** Extrae texto de imágenes o documentos escaneados, convirtiéndolo en datos editables.

Técnicas Clásicas de Procesamiento de Imagen

- **Detección de Objetos:** Identifica y localiza múltiples elementos dentro de una imagen, como coches, personas o animales. Algoritmos como YOLO (*You Only Look Once*) destacan por su eficiencia en tiempo real.
- Se generan **bounding boxes** (cajas delimitadoras) alrededor de los objetos detectados y se clasifican en categorías predefinidas (por ejemplo, personas, coches, animales).
- Ejemplo: Detectar un coche en una imagen y dibujar un rectángulo alrededor de él.

- **Segmentación de Imágenes:** Divide la imagen en regiones significativas, como diferenciar entre fondo y objeto principal. Es clave en tareas como análisis médico.